

TASK 9: Main project - Using ansible

Description

Instructions:

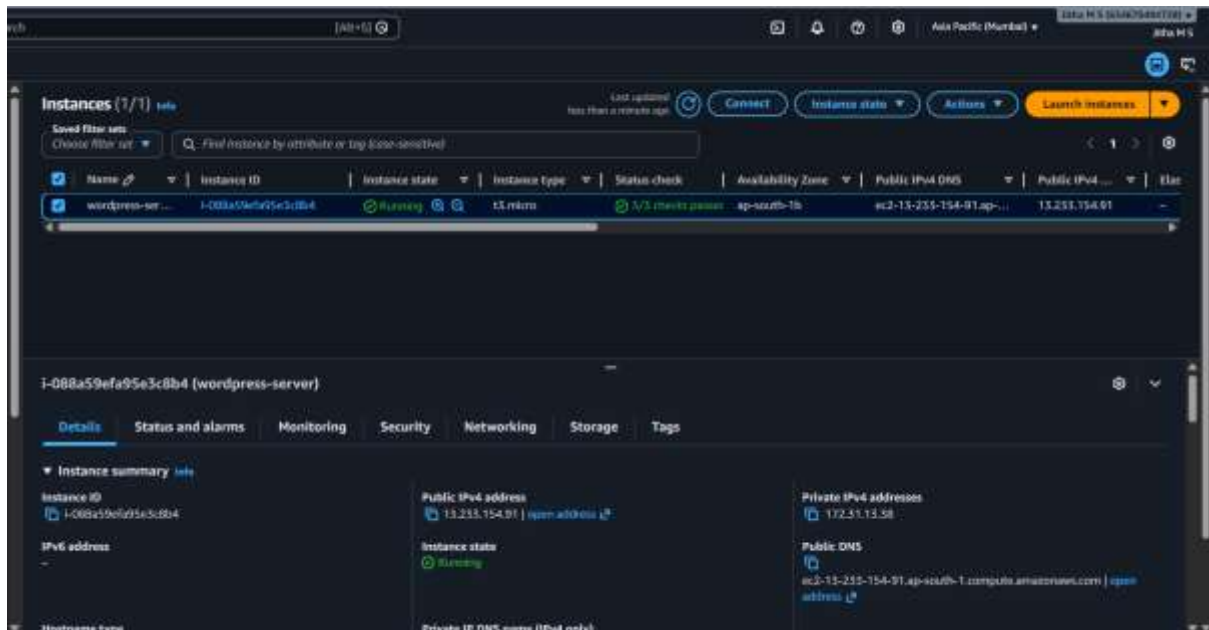
1. Install the necessary packages, which include MySQL 10.6, PHP version 8 or higher, and Nginx on your Linux server.
2. Create a user's home directory under the '/home' directory. You should set the root directory for the website to be '/home/username/website/public.'
3. Configure SFTP (Secure File Transfer Protocol) access for the user to ensure secure file management.
4. Configure phpmyadmin access for the user to ensure secure database management.
4. Install a free SSL certificate to enable secure HTTPS access to your website.
5. Once the website is hosted, proceed to the WordPress dashboard to make necessary configurations and customizations.
6. Write a personal blog post about yourself using the WordPress content management system.

Objective:

This project demonstrates the deployment of a WordPress website on an Amazon Linux 2023 server using AWS EC2. The project includes the installation and configuration of Nginx, PHP, MariaDB, phpMyAdmin, SFTP, and SSL services. The objective is to create a secure and fully functional WordPress hosting environment.

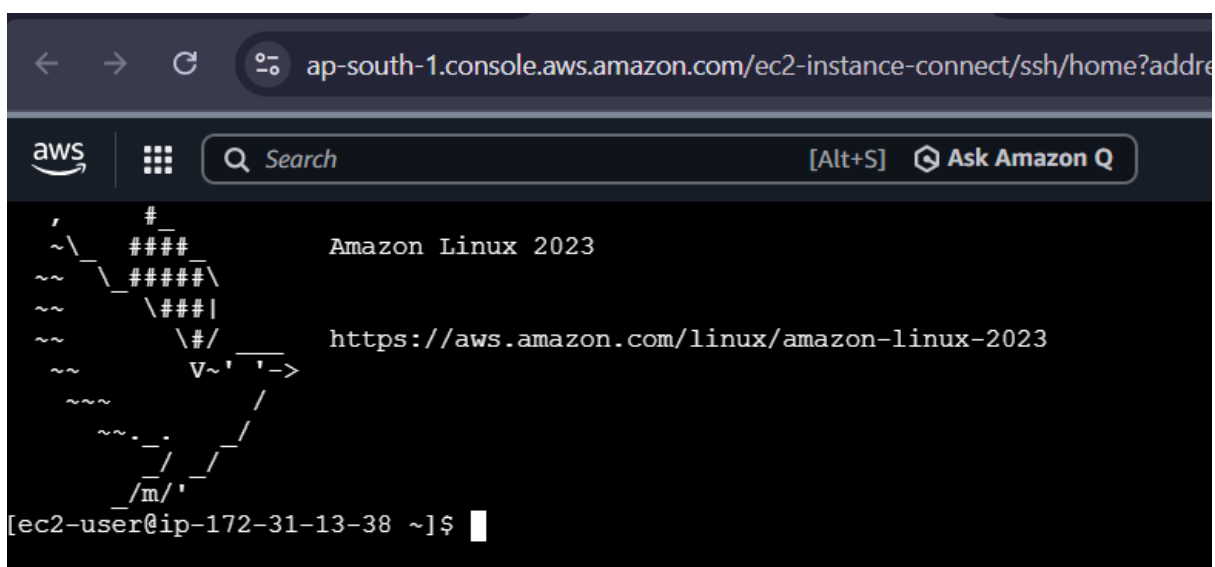
Step 1: EC2 Instance Running State

The EC2 instance was successfully launched and verified in the running state. The public IP address was used for server access and website hosting.



Step 2: SSH Connection to EC2

A secure SSH connection was established from the local system to the EC2 instance. This allowed remote administration and configuration of the server.



Step 3: System Update

The server packages were updated to the latest available versions using the DNF package manager. This ensured that the system was secure and ready for software installation.

```
[ec2-user@ip-172-31-13-38 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository
Amazon Linux 2023 Kernel Livepatch repository
3
99 kB/s | 47 kB    00:00
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-13-38 ~]$
```

Step 4: Install Ansible

Ansible was installed on the Amazon Linux server to automate configuration and deployment tasks. The installation was verified by checking the installed Ansible version.

```
Installing : ansible-8.3.0-1.amzn2023.0.2.noarch [
Installing : ansible-8.3.0-1.amzn2023.0.2.noarch [
Installing : ansible-8.3.0-1.amzn2023.0.2.noarch [
Installing : ansible-8.3.0-1.amzn2023.0.2.noarch [
4/4
Running scriptlets: ansible-8.3.0-1.amzn2023.0.2.noarch
4/4
Verifying  : ansible-8.3.0-1.amzn2023.0.2.noarch
1/4
Verifying  : ansible-core-2.15.3-1.amzn2023.0.1.x86_64
2/4
Verifying  : git-core-2.56.1-1.amzn2023.0.1.x86_64
3/4
Verifying  : sshpass-1.09-6.amzn2023.0.1.x86_64
4/4
Installed:
ansible-8.3.0-1.amzn2023.0.2.noarch      ansible-core-2.15.3-1.amzn2023.0.1.x86_64      git-core-2.56.1-1.amzn2023.0.1.x86_64      sshpass-1.09-6.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-172-31-13-38 ~]$ ansible --version
ansible [core 2.15.3]
  config file = None
  configured module search path = ['/home/ec2-user/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3.9/site-packages/ansible
  ansible collection location = /home/ec2-user/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.9.25 (main, May 23 2026, 00:00:00) (GCC 11.5.0 20240719 (Red Hat 11.5.0-5)) (/usr/bin/python3.9)
  libyaml = True
[ec2-user@ip-172-31-13-38 ~]$
```

Step 5: Create Ansible Project Directory

A dedicated project directory was created to store Ansible configuration files, inventory files, and playbooks. This helped organize the automation project efficiently.

```
[ec2-user@ip-172-31-13-38 ~]$ mkdir wordpress-project
[ec2-user@ip-172-31-13-38 ~]$ cd wordpress-project
[ec2-user@ip-172-31-13-38 wordpress-project]$ pwd
/home/ec2-user/wordpress-project
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

Step 6: Create Inventory File

An inventory file was created to define the target host for Ansible automation. The inventory acts as a list of servers that Ansible can manage.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ vim inventory
[ec2-user@ip-172-31-13-38 wordpress-project]$ cat inventory
[web]
localhost ansible_connection=local
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

Step 7: Create Ansible Playbook

An Ansible playbook was created to automate the installation and configuration of required services. The playbook contains tasks that simplify server setup and deployment.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ vim wordpress.yml
[ec2-user@ip-172-31-13-38 wordpress-project]$ cat wordpress.yml
---
- hosts: web
  become: yes

  tasks:

    - name: Install Nginx
      dnf:
        name: nginx
        state: present

    - name: Install PHP
      dnf:
        name:
          - php
          - php-fpm
          - php-mysqlnd
        state: present

    - name: Install MariaDB
      dnf:
        name:
          - mariadb105-server
        state: present

    - name: Start Nginx
      service:
        name: nginx
        state: started
        enabled: yes

    - name: Start MariaDB
      service:
        name: mariadb
        state: started
        enabled: yes
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

Step 8: Execute Ansible Playbook

The Ansible playbook was executed against the target server. Successful execution ensured that all required packages and configurations were applied automatically.

```

[ec2-user@ip-172-31-13-38 wordpress-project] $ ansible-playbook -i inventory wordpress.yml
PLAY [web] *************************************************************************************************************************************
TASK [Gathering Facts] *************************************************************************************************************************
[WARNING]: Platform linux on host localhost is using the discovered Python interpreter at /usr/bin/python3.9, but future installations of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
[0]: [localhost]
TASK [install mysql] ********************************************************************************
changed: [localhost]
TASK [install php] ********************************************************************************
changed: [localhost]
TASK [install mariadb] ********************************************************************************
changed: [localhost]
TASK [start mysql] ********************************************************************************
changed: [localhost]
TASK [start mariadb] ********************************************************************************
changed: [localhost]
PLAY RECAP ******************************************************************************************
localhost                  : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[ec2-user@ip-172-31-13-38 wordpress-project] $

```

Step 9: Create Website User

A dedicated user account named **alice** was created to manage website files securely. A password was configured for authentication and SFTP access.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo useradd alice
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo passwd alice
Changing password for user alice.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

Step 10: Create Website Directory Structure

The website root directory was created under `/home/alice/myblog/public` as specified in the project requirements. This directory serves as the document root for the WordPress website.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo mkdir -p /home/alice/myblog/public
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo chown -R alice:alice /home/alice/myblog
[ec2-user@ip-172-31-13-38 wordpress-project]$ ls -l /home/alice
ls: cannot open directory '/home/alice': Permission denied
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo ls -l /home/alice
total 0
drwxr-xr-x. 3 alice alice 20 Jun 10 06:44 myblog
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo ls -l /home/alice/myblog/public
total 0
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

Step 11: Create Nginx Virtual Host

A custom Nginx virtual host configuration was created for hosting the WordPress website. The configuration specifies the domain name, document root, and PHP processing settings.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo vim /etc/nginx/conf.d/myblog.conf
[ec2-user@ip-172-31-13-38 wordpress-project]$ cat sudo vim /etc/nginx/conf.d/myblog.conf
cat: sudo: No such file or directory
cat: vim: No such file or directory
server {

listen 80;

server_name yourdomain.com www.yourdomain.com;

root /home/alice/myblog/public;

index index.php index.html;

location / {
try_files $uri $uri/ /index.php?$args;
}

location ~ \.php$ {
fastcgi_pass unix:/run/php-fpm/www.sock;
include fastcgi_params;
fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
}
}

[ec2-user@ip-172-31-13-38 wordpress-project]$
```

Step 12: Download WordPress

The latest version of WordPress was downloaded from the official WordPress website. The package contains all files required to deploy the content management system.

[illegible]

Step 13: Extract and Copy WordPress Files to Website Root

The downloaded WordPress archive was extracted and copied to the website root directory. Appropriate permissions were configured to allow access by the web server.

```
wordpress/wp-includes/widgets/class-wp-widget-archives.php
wordpress/wp-includes/widgets/class-wp-widget-block.php
wordpress/wp-includes/widgets/class-wp-widget-calendar.php
wordpress/wp-includes/widgets/class-wp-widget-categories.php
wordpress/wp-includes/widgets/class-wp-widget-custom-html.php
wordpress/wp-includes/widgets/class-wp-widget-links.php
wordpress/wp-includes/widgets/class-wp-widget-media-audio.php
wordpress/wp-includes/widgets/class-wp-widget-media-gallery.php
wordpress/wp-includes/widgets/class-wp-widget-media-image.php
wordpress/wp-includes/widgets/class-wp-widget-media-video.php
wordpress/wp-includes/widgets/class-wp-widget-media.php
wordpress/wp-includes/widgets/class-wp-widget-meta.php
wordpress/wp-includes/widgets/class-wp-widget-pages.php
wordpress/wp-includes/widgets/class-wp-widget-recent-comments.php
wordpress/wp-includes/widgets/class-wp-widget-recent-posts.php
wordpress/wp-includes/widgets/class-wp-widget-rss.php
wordpress/wp-includes/widgets/class-wp-widget-search.php
wordpress/wp-includes/widgets/class-wp-widget-tag-cloud.php
wordpress/wp-includes/widgets/class-wp-widget-text.php
wordpress/wp-includes/widgets.php
wordpress/wp-includes/wp-db.php
wordpress/wp-includes/wp-diff.php
wordpress/wp-links-opml.php
wordpress/wp-load.php
wordpress/wp-login.php
wordpress/wp-mail.php
wordpress/wp-settings.php
wordpress/wp-signup.php
wordpress/wp-trackback.php
wordpress/xmlrpc.php
[ec2-user@ip-172-31-13-38 tmp]$ sudo cp -r wordpress/* /home/alice/myblog/public/
[ec2-user@ip-172-31-13-38 tmp]$
```

Step 14: Create WordPress Database

A MariaDB database was created to store WordPress content and configuration data. A dedicated database user was also configured with the required privileges.

```
[ec2-user@ip-172-31-13-38 tmp]$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE wordpress;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]>
MariaDB [(none)]> CREATE USER 'wpuser'@'localhost'
    -> IDENTIFIED BY 'Password@123';
Query OK, 0 rows affected (0.004 sec)

MariaDB [(none)]>
MariaDB [(none)]> GRANT ALL PRIVILEGES
    -> ON wordpress.*
    -> TO 'wpuser'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]>
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]>
MariaDB [(none)]> EXIT;
```

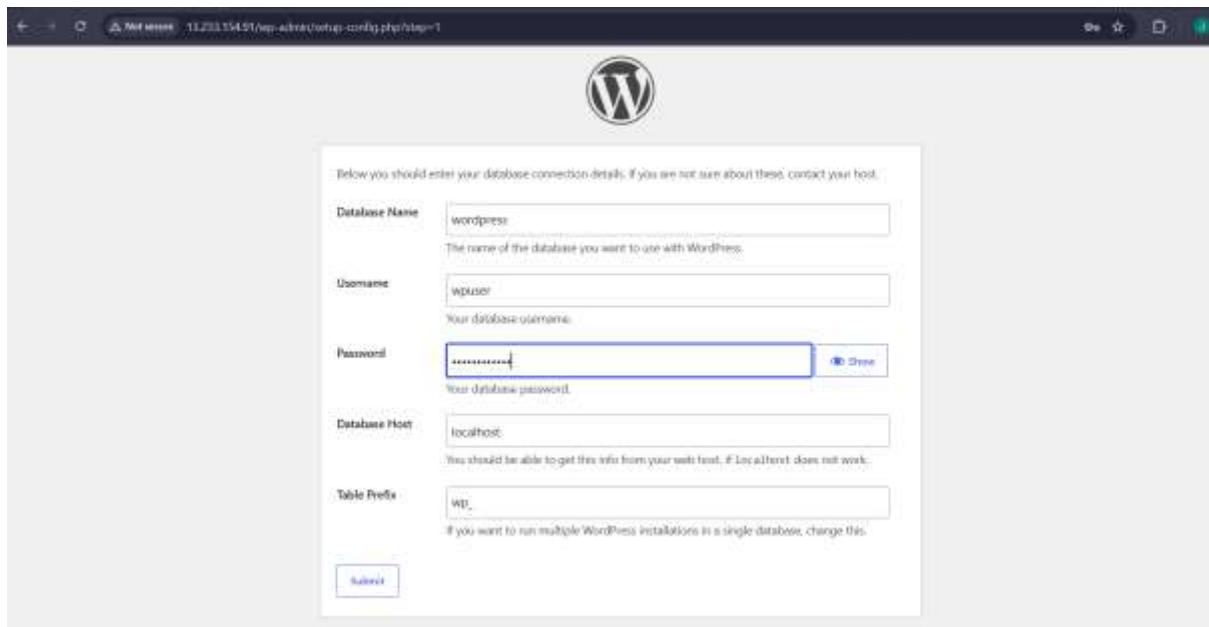
Step 15: Configure wp-config.php

The WordPress configuration file (wp-config.php) was updated with the database connection details. This allows WordPress to communicate with the MariaDB database server.



Step 16: WordPress Database Configuration

Database credentials including database name, username, password, and host information were entered during the WordPress setup process. This completed the database connectivity configuration.



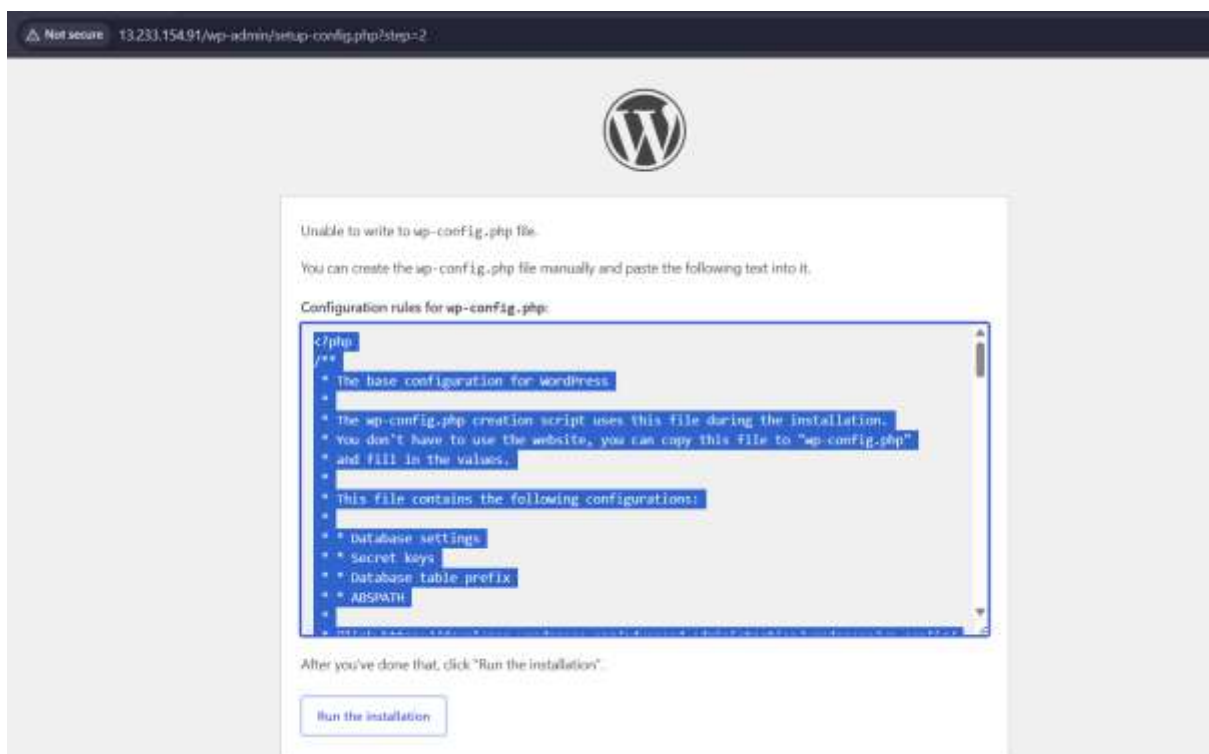
The screenshot shows the WordPress installation screen for database configuration. The URL in the browser is 13.233.154.91/wp-admin/setup-config.php?step=1. The page features the WordPress logo at the top. Below it, a message states: "Below you should enter your database connection details. If you are not sure about these, contact your host." The form contains the following fields:

- Database Name:** wordpress (with a note: "The name of the database you want to use with WordPress.")
- Username:** wpuser (with a note: "Your database username.")
- Password:** [masked] (with a note: "Your database password." and a "Show" button)
- Database Host:** localhost (with a note: "You should be able to get this info from your web host. If localhost does not work.")
- Table Prefix:** WP_ (with a note: "If you want to run multiple WordPress installations in a single database, change this.")

A "Submit" button is located at the bottom left of the form.

Step 17: WordPress Configuration File

The WordPress configuration file was generated and verified successfully. This file contains the database settings and essential application configuration parameters.



The screenshot shows the WordPress installation screen for the configuration file. The URL in the browser is 13.233.154.91/wp-admin/setup-config.php?step=2. The page features the WordPress logo at the top. Below it, a message states: "Unable to write to wp-config.php file. You can create the wp-config.php file manually and paste the following text into it." The form contains the following text:

Configuration rules for wp-config.php:

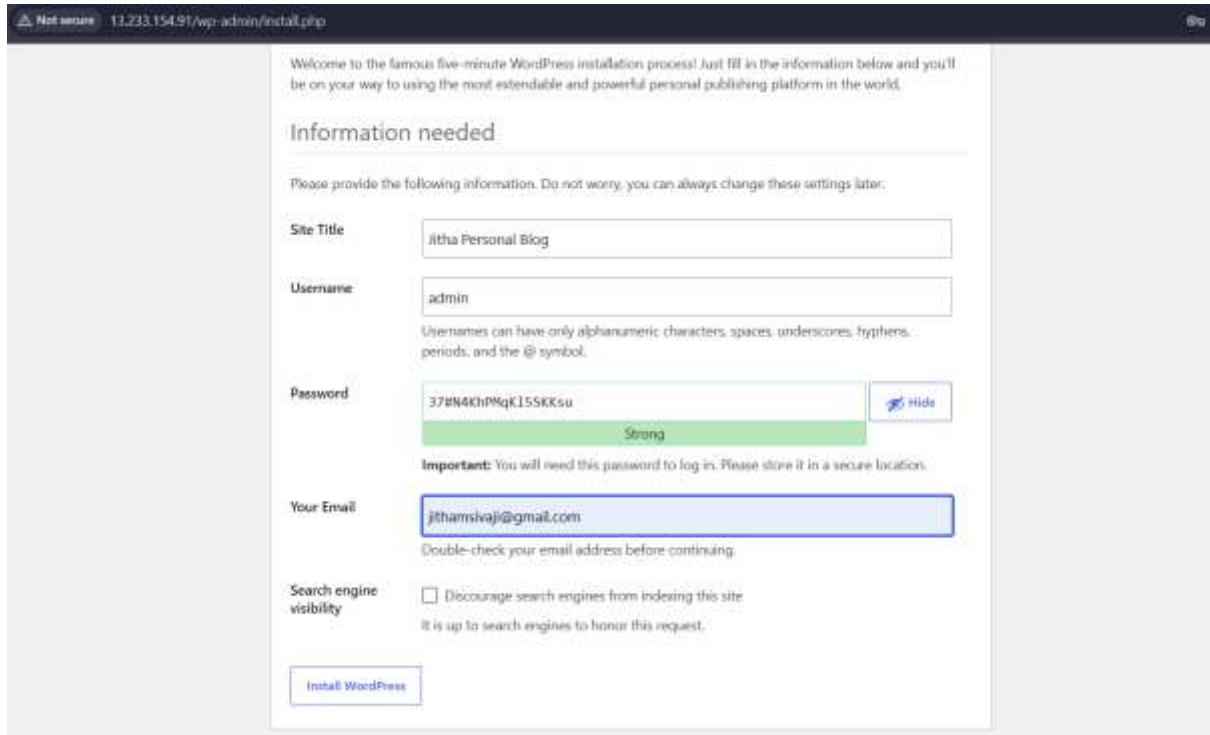
```
c:/php
/**
 * The base configuration for WordPress
 *
 * The wp-config.php creation script uses this file during the installation.
 * You don't have to use the website, you can copy this file to "wp-config.php"
 * and fill in the values.
 *
 * This file contains the following configurations:
 *
 * * Database settings
 * * Secret keys
 * * Database table prefix
 * * ABSPATH
 */
```

After you've done that, click "Run the installation".

A "Run the installation" button is located at the bottom of the page.

Step 18: WordPress Installation Page

The WordPress installation wizard was successfully accessed through a web browser. Site information and administrator account details were configured during this step.



This screenshot shows the 'Information needed' step of the WordPress installation process. The browser's address bar indicates the URL '13.233.154.91/wp-admin/install.php'. The page features a welcome message and a list of fields to be filled out: 'Site Title' (jitha Personal Blog), 'Username' (admin), 'Password' (37#N4KhPMqK155KKsu, marked as Strong), and 'Your Email' (jithamsvaji@gmail.com). There is also a checkbox for 'Search engine visibility' which is currently unchecked. An 'Install WordPress' button is located at the bottom of the form.

Not secure 13.233.154.91/wp-admin/install.php

Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.

Information needed

Please provide the following information. Do not worry, you can always change these settings later.

Site Title

Username
Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.

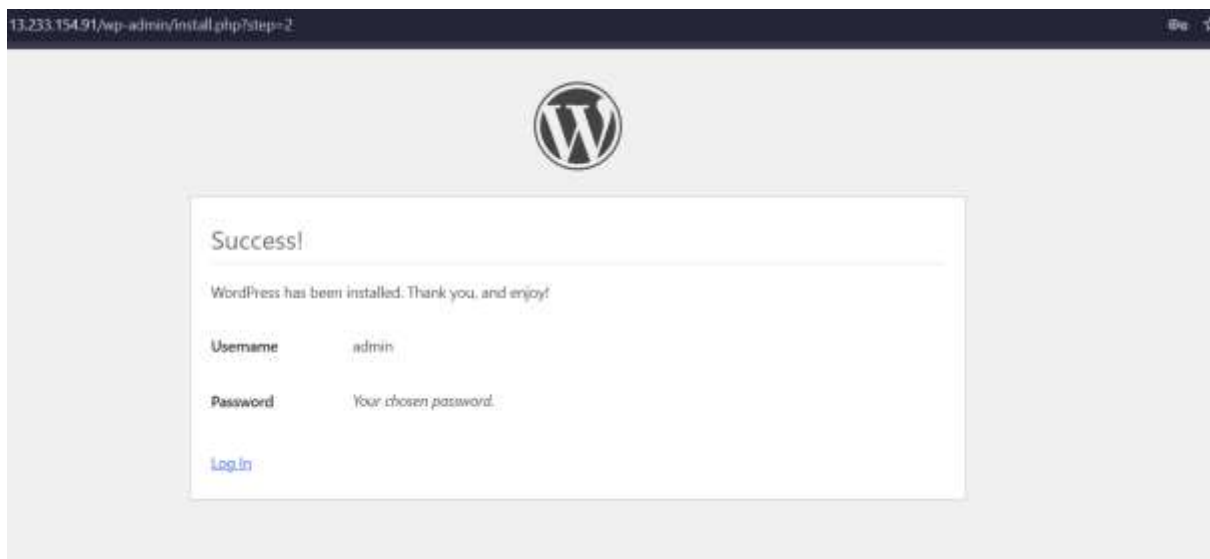
Password [Hide](#)
Strong

Important: You will need this password to log in. Please store it in a secure location.

Your Email
Double-check your email address before continuing.

Search engine visibility ☐ Discourage search engines from indexing this site
It is up to search engines to honor this request.

[Install WordPress](#)



This screenshot shows the 'Success!' page after the WordPress installation is complete. The browser's address bar shows '13.233.154.91/wp-admin/install.php?step=2'. The page features the WordPress logo and a message stating 'WordPress has been installed. Thank you, and enjoy!'. Below this, the 'Username' is listed as 'admin' and the 'Password' is listed as 'Your chosen password.'. A 'Log in' link is provided at the bottom.

13.233.154.91/wp-admin/install.php?step=2



Success!

WordPress has been installed. Thank you, and enjoy!

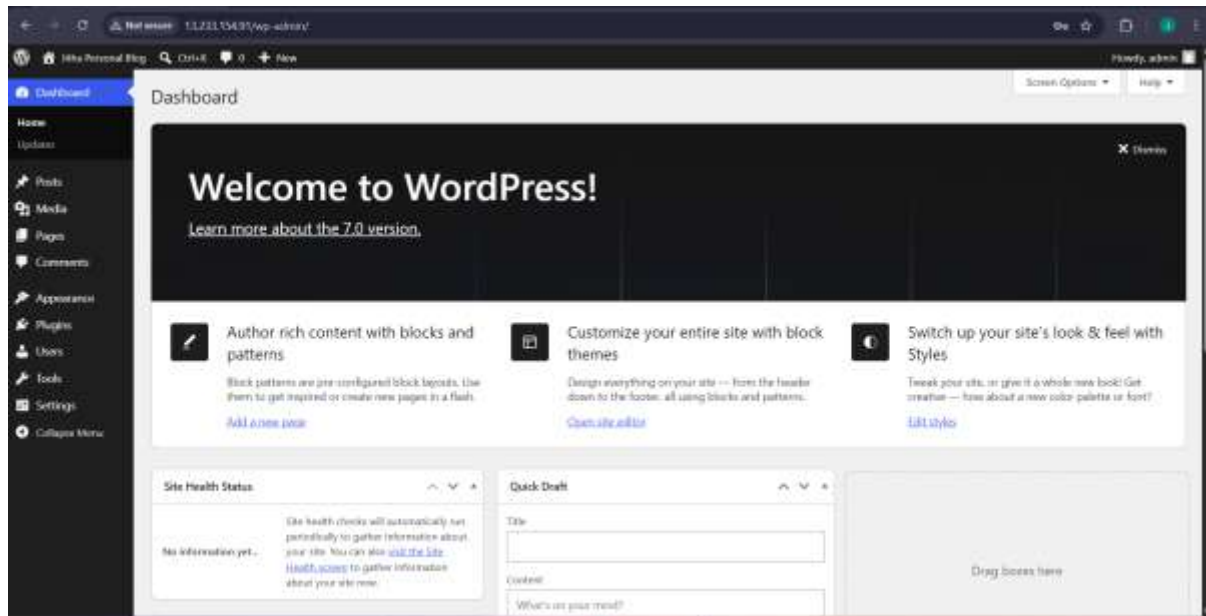
Username admin

Password Your chosen password.

[Log in](#)

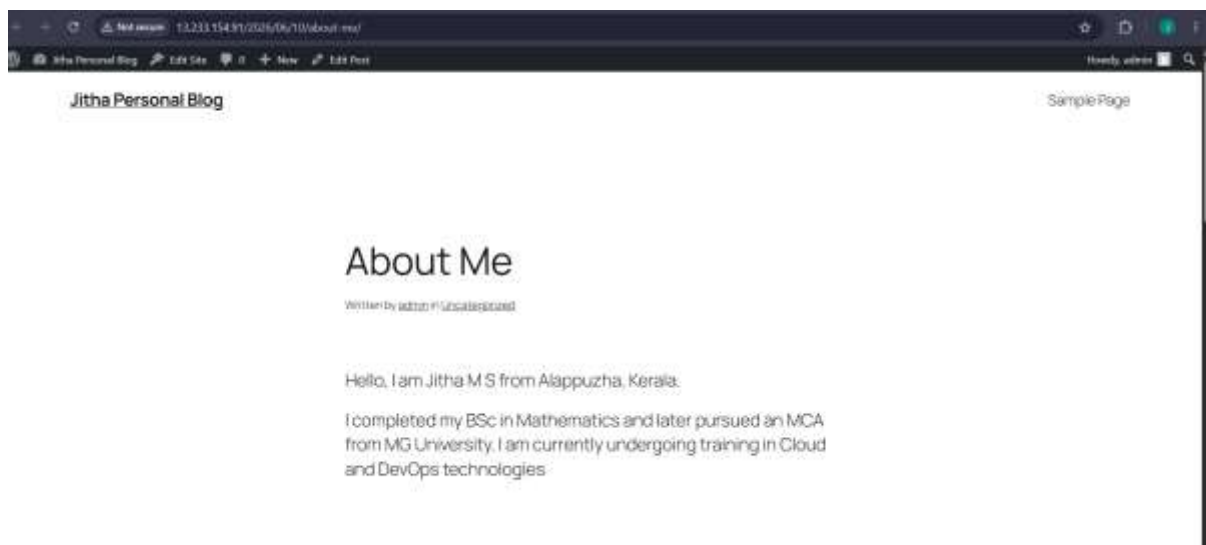
Step 19: WordPress Dashboard

After successful installation, the WordPress administrative dashboard was accessed. The dashboard provides complete control over website content, appearance, and settings.



Step 20: Create Blog Post

A personal blog post was created and published using the WordPress content management system. This verified that the website was functioning correctly and ready for content publishing.



Step 21: Download phpMyAdmin

The latest phpMyAdmin package was downloaded from the official website. This tool provides a graphical interface for managing MySQL and MariaDB databases.

```
luc2-user@ip-172-31-13-38 tmp]$ wget https://www.phpmyadmin.net/downloads/phpmyadmin-latest-all-languages.tar.gz
--2024-06-10 07:23:13-- https://www.phpmyadmin.net/downloads/phpmyadmin-latest-all-languages.tar.gz
Resolving www.phpmyadmin.net (www.phpmyadmin.net)... 89.187.162.12, 89.187.163.20, 79.127.235.11, ...
Connecting to www.phpmyadmin.net (www.phpmyadmin.net):80.187.162.12:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://files.phpmyadmin.net/phpMyAdmin/5.2.3/phpMyAdmin-5.2.3-all-languages.tar.gz [following]
--2024-06-10 07:23:14-- https://files.phpmyadmin.net/phpMyAdmin/5.2.3/phpMyAdmin-5.2.3-all-languages.tar.gz
Resolving files.phpmyadmin.net (files.phpmyadmin.net)... 79.127.235.5, 89.187.162.13, 79.127.235.12, ...
Connecting to files.phpmyadmin.net (files.phpmyadmin.net):79.127.235.5:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 14446385 (14M) [application/octet-stream]
Saving to: 'phpMyAdmin-latest-all-languages.tar.gz'

phpMyAdmin-latest-all-languages.tar.gz 0%
MyAdmin-latest-all-languages.tar.gz 1%
min-latest-all-languages.tar.gz 1%
s-latest-all-languages.tar.gz 55%
latest-all-languages.tar.gz 100%
2024-06-10 07:23:15 (22.4 MB/s) - 'phpMyAdmin-latest-all-languages.tar.gz' saved [14446385/14446385]

luc2-user@ip-172-31-13-38 tmp]$
```

Step 22: Extract phpMyAdmin

The downloaded phpMyAdmin archive was extracted on the server. The extracted files were prepared for deployment within the web server environment.

```
luc2-user@ip-172-31-13-38 tmp$ tar -xzf phpmyadmin-latest-all-languages.tar.gz
luc2-user@ip-172-31-13-38 tmp$ ls
latest-all-languages
phpMyAdmin-5.2.3-all-languages
phpMyAdmin-latest-all-languages.tar.gz
systemd-private-d04c947d8a15414eb5be023ee33b5e0-gnss.service-3h8f9b
systemd-private-d04c947d8a15414eb5be023ee33b5e0-php-fpm.service-cf8f28
systemd-private-d04c947d8a15414eb5be023ee33b5e0-policy-routesens1.service-WFVeh1
systemd-private-d04c947d8a15414eb5be023ee33b5e0-systemd-logind.service-b1TVZ1
systemd-private-d04c947d8a15414eb5be023ee33b5e0-systemd-resolved.service-c5Ub3q
wordpress
```

Step 23: Configure phpMyAdmin

phpMyAdmin was configured and integrated with the Nginx web server. Appropriate settings were applied to allow secure database management through a browser.

```
luc2-user@ip-172-31-13-38 tmp$ ls -la /usr/share/nginx/html/phpmyadmin
CONTRIBUTING.md  LICENSE  RELEASE-DATES-3.3.1  composer.json  config.sample.inc.php  examples  index.php  libraries  package.json  seCup  sql  themes  vendor
CHANGELOG  README  babel.config.json  composer.lock  doc  favicon.ico  js  locale  robots.txt  show_config_errors.php  templates  url.php  yaml
luc2-user@ip-172-31-13-38 tmp$
```

Step 24: Verify Nginx Configuration

The Nginx configuration was tested to ensure there were no syntax or configuration errors. Successful validation confirmed that the web server was correctly configured.

```
[ec2-user@ip-172-31-13-38 tmp]$ sudo chown -R nginx:nginx /usr/share/nginx/html/phpmyadmin
[ec2-user@ip-172-31-13-38 tmp]$ sudo vim /etc/nginx/conf.d/phpmyadmin.conf
[ec2-user@ip-172-31-13-38 tmp]$ cat sudo vim /etc/nginx/conf.d/phpmyadmin.conf
cat: sudo: No such file or directory
cat: vim: No such file or directory
location /phpmyadmin {
    root /usr/share/nginx/html;
    index index.php;
}

location ~ ^/phpmyadmin/(.+\.php)$ {
    root /usr/share/nginx/html;
    fastcgi_pass unix:/run/php-fpm/www.sock;
    include fastcgi_params;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
}
[ec2-user@ip-172-31-13-38 tmp]$
```

Step 25: Access phpMyAdmin

The phpMyAdmin web interface was successfully accessed through the browser. This confirmed that the installation and configuration were completed successfully.



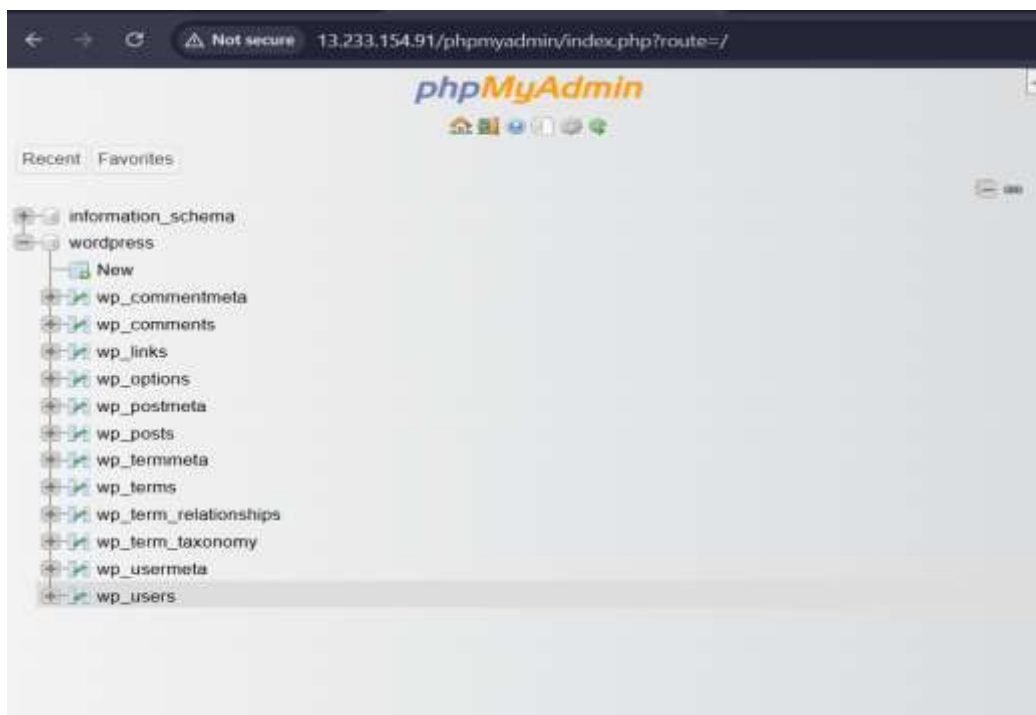
Step 26: Login to phpMyAdmin

Authentication was performed using the configured database user credentials. Successful login verified database connectivity and user permissions.



Step 27: phpMyAdmin Dashboard

The phpMyAdmin dashboard was accessed successfully. The available databases and management options were verified through the graphical interface.



Step 28: SFTP Configuration

SFTP access was configured for the user **alice** to enable secure file transfer and website management. This ensures secure communication between the client and server.

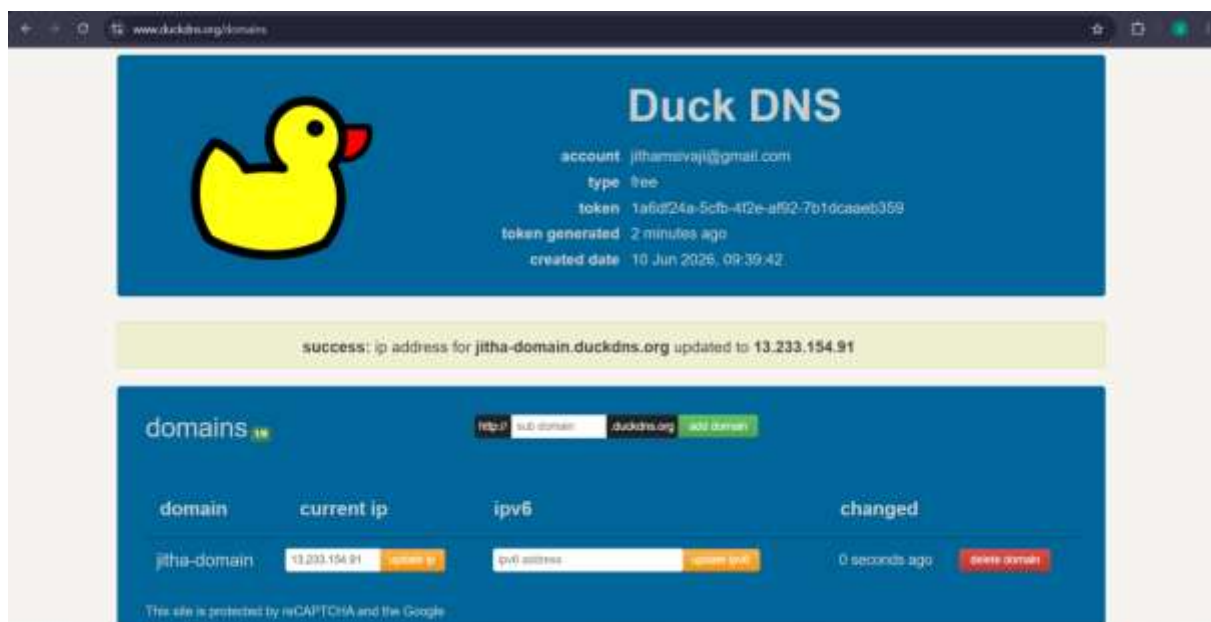
```
Amazon Linux 2023
https://www.amazon.com/linux/amazon-linux-2023

Last login: Wed Jun 10 06:15:39 2026 from 13.233.177.4
[ec2-user@ip-172-31-13-38 ~]$ sudo systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-06-10 06:13:23 UTC; 1h 18min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
    Main PID: 11037 (sshd)
      Tasks: 1 (limit: 1014)
     Memory: 8.1M
        CPU: 867ms
    CGroup: /system.slice/ssh.service
            └─11037 "sshd /usr/sbin/sshd -D [listener] 3 of 10-100 startups"

Jun 10 06:13:23 ip-172-31-13-38.ap-south-1.compute.internal systemd[1]: Starting sshd.service - OpenSSH server daemon...
Jun 10 06:13:23 ip-172-31-13-38.ap-south-1.compute.internal sshd[11037]: Server listening on 0.0.0.0 port 22.
Jun 10 06:13:23 ip-172-31-13-38.ap-south-1.compute.internal sshd[11037]: Server listening on :: port 22.
Jun 10 06:13:23 ip-172-31-13-38.ap-south-1.compute.internal systemd[1]: Started sshd.service - OpenSSH server daemon.
Jun 10 09:26:36 ip-172-31-13-38.ap-south-1.compute.internal ec2-instance-connect[37263]: Querying EC2 Instance Connect Keys for matching fingerprint: 38A254:88
Jun 10 09:26:36 ip-172-31-13-38.ap-south-1.compute.internal sshd[37117]: Accepted publickey for ec2-user from 13.233.177.4 port 35673 ssh2: ec25519 ssh256:888
Jun 10 09:26:36 ip-172-31-13-38.ap-south-1.compute.internal sshd[37117]: pam_unix(sshd:session): session opened for user ec2-user(uid=1000) by (uid=0)
[ec2-user@ip-172-31-13-38 ~]$ getent passwd alice
-bash: getent: command not found
[ec2-user@ip-172-31-13-38 ~]$ getent passwd alice
alice:x:1001:1001::/home/alice:/bin/bash
[ec2-user@ip-172-31-13-38 ~]$
```

Step 29: Domain DNS Configuration

A DuckDNS domain was created and configured to point to the EC2 instance public IP address. DNS resolution was verified to ensure proper domain connectivity.



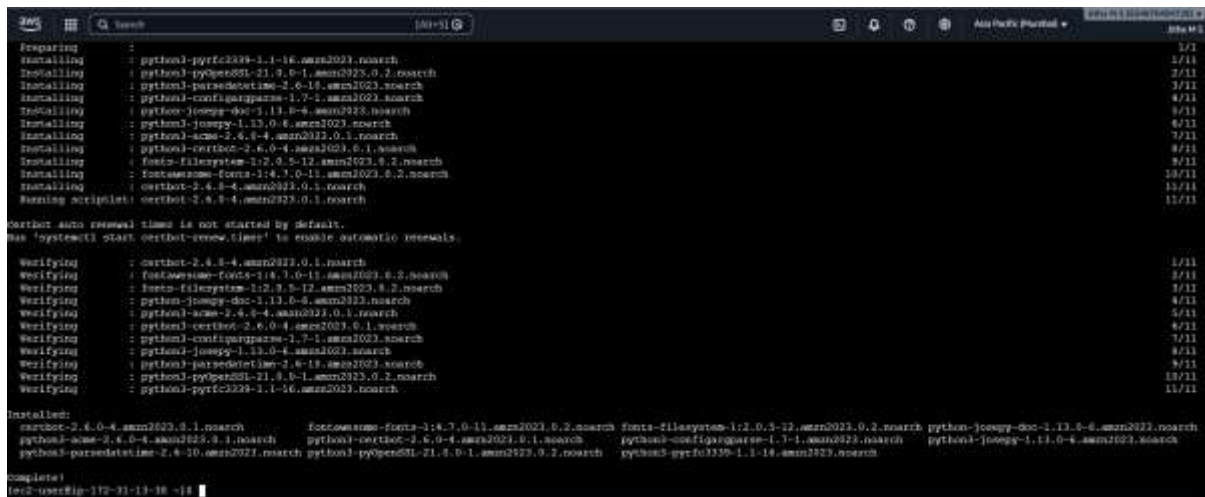
Step 30: WordPress Website Access Verification Using Domain Name

The WordPress website was successfully accessed using the configured domain name. This confirmed that DNS records and web server settings were functioning correctly.



Step 31: Install Certbot

Certbot was installed to obtain and manage free SSL certificates from Let's Encrypt. This tool simplifies SSL certificate generation and renewal.



Step 32: HTTPS Verification

The website was tested using HTTPS to verify secure communication. Browser verification confirmed that SSL encryption was functioning correctly.

```
[ec2-user@ip-172-31-13-30 ~]$ sudo certbot certonly --standalone --register-unsafely-without-email -d jitha-domain.duckdns.org
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Requesting a certificate for jitha-domain.duckdns.org

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/jitha-domain.duckdns.org/fullchain.pem
Key is saved at: /etc/letsencrypt/live/jitha-domain.duckdns.org/privkey.pem
This certificate expires on 2026-09-08.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

-----
If you like Certbot, please consider supporting our work by:
 * Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
 * Donating to EFF: https://eff.org/donate-le
-----
[ec2-user@ip-172-31-13-30 ~]$
```

Step 33: Generate SSL Certificate

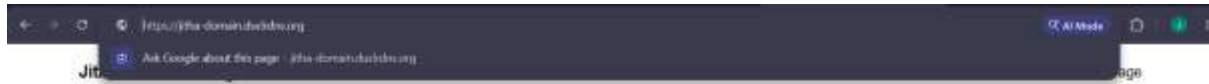
A free SSL certificate was successfully generated and installed using Certbot. The certificate was linked to the configured domain name and web server.

```
Jun 10 10:07:56 ip-172-31-13-30.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-172-31-13-30 ~]$ sudo certbot certificates
Saving debug log to /var/log/letsencrypt/letsencrypt.log

-----
Found the following certs:
Certificate Name: jitha-domain.duckdns.org
Serial Number: 58962ba49ff2e2155c170c7f1daac72987d
Key Type: ECDSA
Domains: jitha-domain.duckdns.org
Expiry Date: 2026-09-08 09:01:00+00:00 (VALID: 89 days)
Certificate Path: /etc/letsencrypt/live/jitha-domain.duckdns.org/fullchain.pem
Private Key Path: /etc/letsencrypt/live/jitha-domain.duckdns.org/privkey.pem
-----
[ec2-user@ip-172-31-13-30 ~]$
```

Step 34: HTTPS Access Verification After SSL Installation

The website was successfully accessed using HTTPS after SSL configuration. The secure connection verified that encrypted communication was enabled for all users.



Conclusion:

This project successfully demonstrates the deployment of a WordPress website on Amazon Linux 2023 using AWS EC2 and Ansible. Nginx, PHP, MariaDB, phpMyAdmin, SFTP, domain configuration, and SSL security were implemented successfully. The website was verified through both HTTP and HTTPS access, and a personal blog post was published to confirm complete functionality.